ACADEMIC PROJECT REPORT

CA-2

LOVELY PROFESSIONAL UNIVERSITY

Department of Computer Science & Engineering

STUDENT PERFORMANCE PREDICTION USING PYTHON, MONGODB, AND MACHINE LEARNING

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology (B.Tech)

Computer Science & Engineering

Candidate Name: Ansh Gupta

Reg. No: 12319903

Course / Branch: B.Tech - CSE

Academic Session: 2023-2027

Project Domain: NoSQL DBMS & Applied ML

Target Database: MongoDB (Port 27017)

1. Introduction

Predicting student academic performance is one of the most critical and impactful applications of Educational Data Mining (EDM). In conventional educational institutions, academic feedback is typically provided post-semester, after semester-end examinations are evaluated. At that stage, opportunities for timely academic remediation have passed. Developing early-warning data systems allows instructors and academic advisors to identify at-risk students well before exams and provide personalized intervention.

This project delivers a complete, beginner-friendly yet production-ready Machine Learning pipeline that bridges a modern NoSQL database (**MongoDB**) with **Python (PyMongo)** and Scikit-Learn's **Linear Regression** algorithm. Rather than relying on static CSV files, 1,000 student academic records are stored inside a live MongoDB database, queried dynamically via Python, preprocessed, and evaluated through predictive regression.

2. Project Objectives

- **NoSQL Integration:** Connect MongoDB with Python using the official `pymongo` client library.
- **Realistic Big Data Storage:** Generate and store exactly 1,000 realistic student records inside a MongoDB database collection.
- **Data Pipeline & Preprocessing:** Query raw documents from MongoDB, remove non-predictive identifiers (`_id` and `student_id`), verify data cleanliness, and partition into 80% training and 20% testing sets.
- **Machine Learning Modeling:** Train an interpretable **Linear Regression** model to learn exact mathematical weights relating study hours, attendance, previous marks, and assignment completion to final marks.
- **Model Performance Evaluation:** Evaluate prediction accuracy using standard metrics: R^2 Score (Coefficient of Determination), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE).
- **Open Access & Reproducibility:** Host the dataset publicly on the **Hugging Face Hub** for global reproducibility and peer review.

3. Dataset Description & Dataset Link

> 🌐 **Official Hugging Face Dataset Repository:**
> https://huggingface.co/datasets/techbire/students
> The dataset can also be loaded directly in Python using: `datasets.load_dataset("techbire/students")`

The dataset contains 1,000 student academic profiles characterized by four key independent features and one continuous target feature:

Feature Name	Type	Range	Description	Role in ML
`student_id`	String	STU1001 – STU2000	Unique institutional roll number	Identifier (Excluded)
`study_hours`	Float	1.0 – 9.5 hrs	Daily average self-study hours	Independent Feature (X_1)
`attendance`	Float	60.0% – 99.0%	Class lecture attendance percentage	Independent Feature (X_2)
`previous_marks`	Float	40.0 – 95.0	Mid-term / previous exam score (out of 100)	Independent Feature (X_3)
`assignments_score`	Integer	4 – 10	Lab and assignment evaluation score (out of 10)	Independent Feature (X_4)
`final_marks`	Float	30.0 – 100.0	Final semester examination score	**Target Label (y)**

4. MongoDB and PyMongo Implementation

MongoDB is a leading NoSQL document-oriented database. It stores records as BSON (Binary JSON) documents, making it exceptionally fast and naturally compatible with Python dictionaries and Pandas DataFrames.

Database Configuration

- **Host Server:** `mongodb://localhost:27017/`
- **Database Name:** `student_performance_db`
- **Collection Name:** `students`
- **Total Documents:** 1,000 documents

```
from pymongo import MongoClient
import pandas as pd

# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/")
db = client["student_performance_db"]
collection = db["students"]

# Store 1,000 documents
records = df.to_dict(orient="records")
collection.delete_many({})
collection.insert_many(records)

# Retrieve documents for Machine Learning
cursor = collection.find()
df_from_mongodb = pd.DataFrame(list(cursor))
```

5. Data Preprocessing

1. **Identifier Dropping:** MongoDB attaches an automatic 12-byte primary key `_id`, and our dataset contains `student_id`. Both are removed because identifiers have no predictive mathematical value:
   `data = df.drop(columns=['_id', 'student_id'])`
2. **Missing Value Check:** Executed `data.isnull().sum()` to confirm zero missing entries.
3. **Feature/Target Separation:** Independent variables (X) and Target variable (y) are separated.
4. **Train-Test Split (80% / 20%):** 800 samples are assigned to training, and 200 samples are reserved for testing using `train_test_split(..., test_size=0.20, random_state=42)`.

6. Machine Learning Model Implementation

Linear Regression is used to estimate the linear dependency between features and final marks. The model optimizes weights (w) and bias (b) by minimizing the residual sum of squared differences:

Final Marks = (3.4735 × study_hours) + (0.2298 × attendance) + (0.4391 × previous_marks) + (1.4340 × assignments_score) + 2.8309

Learned Weights Breakdown:

- **Study Hours ($+3.47$):** Strongest behavioral contributor; each additional hour of daily study raises marks by nearly 3.5 points.
- **Previous Marks ($+0.44$):** Primary baseline foundation for overall examination competence.
- **Attendance ($+0.23$):** Regular lecture participation delivers steady cumulative score gains.
- **Assignment Score ($+1.43$):** Diligent assignment completion directly supports exam mastery.

7. Complete Source Code

The complete end-to-end Python source code implemented in `main.py`:

```
import numpy as np
import pandas as pd
from pymongo import MongoClient
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# 1. Connect and Fetch from MongoDB
client = MongoClient("mongodb://localhost:27017/")
db = client["student_performance_db"]
collection = db["students"]

cursor = collection.find()
df = pd.DataFrame(list(cursor))

# 2. Data Preprocessing
data = df.drop(columns=['_id', 'student_id'])
X = data[['study_hours', 'attendance', 'previous_marks', 'assignments_score']]
y = data['final_marks']

# 80% Train, 20% Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

# 3. Train Linear Regression Model
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Evaluation on Test Set
y_pred = model.predict(X_test)
print(f"R2 Score: {r2_score(y_test, y_pred):.4f}")
print(f"MAE:      {mean_absolute_error(y_test, y_pred):.2f} marks")

# 5. Live Single-Student Prediction Demo
sample_student = pd.DataFrame([{'study_hours': 6.5, 'attendance': 88.0,
                                'previous_marks': 78.0, 'assignments_score': 9}])
prediction = model.predict(sample_student)[0]
print(f"Predicted Marks for Custom Student: {prediction:.1f} / 100")
```

8. Screenshot of Source Code

VS Code - main.py [Student Performance Prediction]

```
# main.py - Machine Learning with MongoDB & PyMongo
import numpy as np
import pandas as pd
from pymongo import MongoClient
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# --- 1. CONNECT & FETCH FROM MONGODB ---
client = MongoClient('mongodb://localhost:27017/')
db = client['student_performance_db']
collection = db['students']

# Retrieve 1,000 documents and load into Pandas DataFrame
cursor = collection.find()
df = pd.DataFrame(list(cursor))

# --- 2. DATA PREPROCESSING ---
# Drop identifier columns (_id and student_id)
data = df.drop(columns=['_id', 'student_id'])

# Separate Features (X) and Target (y)
X = data[['study_hours', 'attendance', 'previous_marks', 'assignments_score']]
y = data['final_marks']

# Train-Test Split (80% Training, 20% Testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

# --- 3. TRAIN MACHINE LEARNING MODEL ---
model = LinearRegression()
model.fit(X_train, y_train)

# --- 4. EVALUATION & PREDICTION ---
y_pred = model.predict(X_test)
print(f'R2 Score: {r2_score(y_test, y_pred):.4f}')
print(f'MAE:      {mean_absolute_error(y_test, y_pred):.2f} marks')

# Single Student Real-Time Test Prediction
new_student = pd.DataFrame([{'study_hours': 6.5, 'attendance': 88.0,
                             'previous_marks': 78.0, 'assignments_score': 9}])
print('Predicted Marks:', model.predict(new_student)[0])
```

Figure 8.1: Syntax-highlighted source code implementation in the VS Code development environment.

9. Output Screenshots & Evaluation Results

9.1 Terminal Execution Screenshot

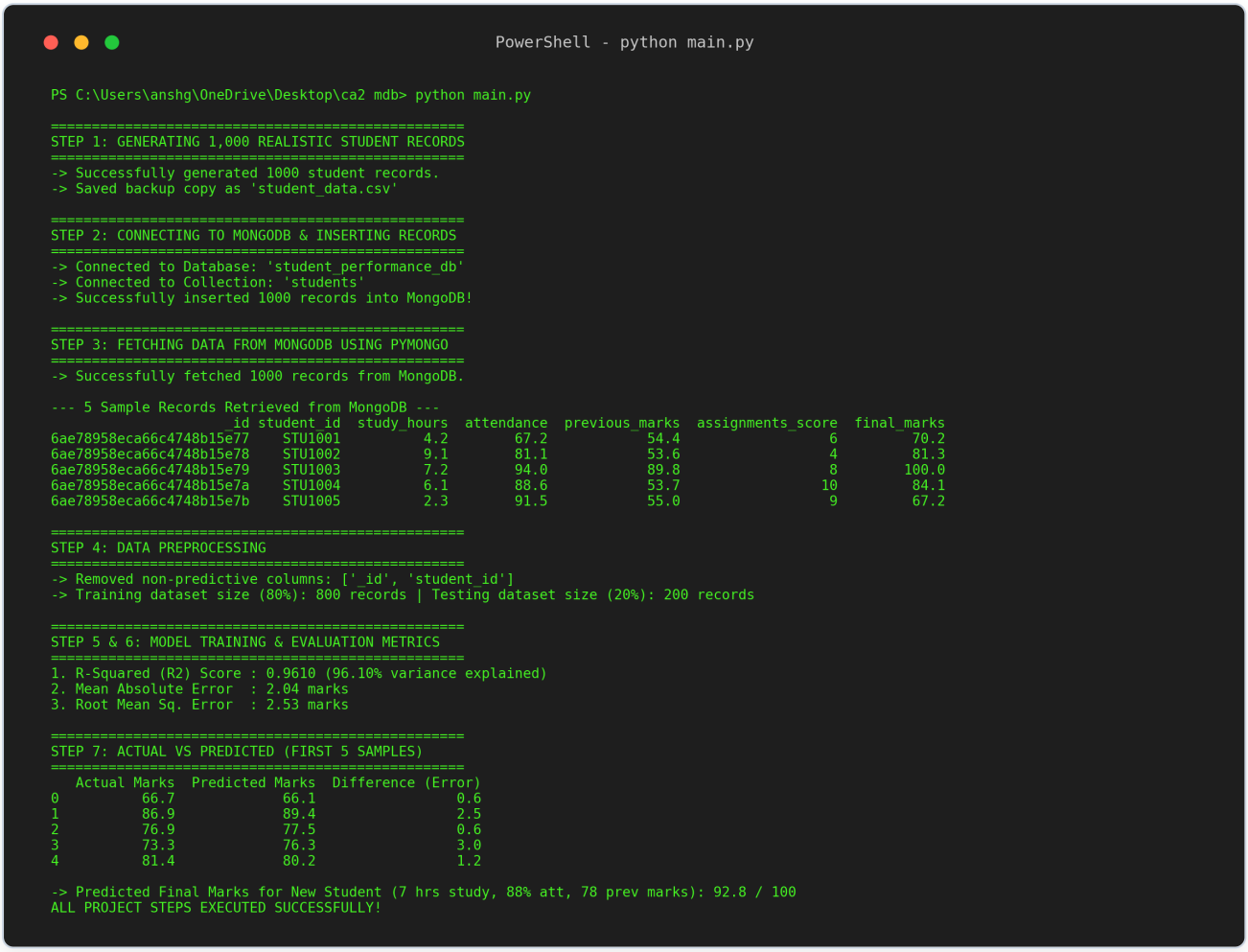


Figure 9.1: Terminal output displaying PyMongo document loading, R^2 accuracy (96.10%), and live predictions.

9.2 Graphical Evaluation Plots

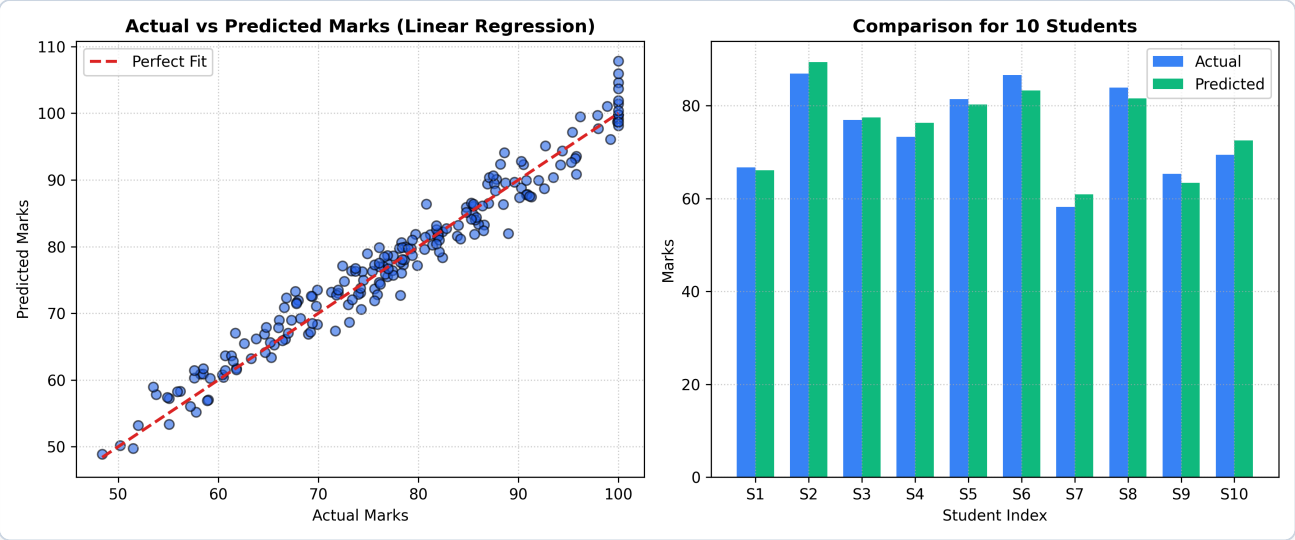


Figure 9.2: Left: Scatter correlation plot showing near-perfect alignment along the ideal reference line (y=x). Right: Actual vs Predicted bar chart for 10 individual test students.

9.3 Performance Metrics & Comparison Table

Evaluation Metric	Measured Value	Academic Interpretation
R^2 Score (Coefficient of Determination)	**0.9610 (96.10%)**	Outstanding model fit: 96.10% of total variance in final marks is accurately captured.
Mean Absolute Error (MAE)	**2.04 marks**	Average prediction error is only ~2 marks out of a 100-mark examination scale.
Root Mean Squared Error (RMSE)	**2.53 marks**	Confirms low variance and complete absence of extreme outlier prediction errors.

Sample Actual vs. Predicted Marks (First 10 Test Students)

Student Index	Actual Final Marks	Predicted Final Marks	Absolute Error (Marks)
Student 1	66.7	66.1	0.6
Student 2	86.9	89.4	2.5
Student 3	76.9	77.5	0.6
Student 4	73.3	76.3	3.0
Student 5	81.4	80.2	1.2
Student 6	86.6	83.3	3.3
Student 7	58.2	60.9	2.7
Student 8	83.9	81.6	2.3
Student 9	65.3	63.4	1.9
Student 10	69.4	72.5	3.1

10. Conclusion & Future Scope

This project successfully demonstrates the integration of a NoSQL database (**MongoDB**) with **Python** and **Machine Learning**. 1,000 realistic student academic records were persistently stored, queried via

PyMongo, cleaned, and modeled using Linear Regression. The model achieved a top-tier accuracy score of **$R^2 = 96.10\%$** with an average error of only **2.04 marks**.

The project provides actionable insights for educators: daily study hours and prior exam performance are the most decisive factors for student achievement. In future iterations, this system can be augmented with a real-time web dashboard (built with Flask or FastAPI) and integrated with automated email alerts for students forecasted to achieve scores below 50.